

АННОТАЦИЯ

Настоящий документ «Пояснительная записка» разработан в соответствии с ГОСТ 19.404–79 и содержит обоснование технических решений, принятых при разработке программного изделия «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (BunnyMQ).

Документ описывает назначение и область применения системы, алгоритмы функционирования сегментированного журнала, конечных автоматов Raft, координаторов и API, а также обоснование выбора технологического стека и ожидаемые показатели производительности.

Документ составлен в соответствии с требованиями ГОСТ 19.105–78 и ГОСТ 19.404–79.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	2
2.1. Назначение программы	2
2.2. Область применения	2
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	3
3.1. Постановка задачи на разработку	3
3.1.1. Описание задачи	3
3.1.2. Описание применяемых методов	3
3.1.3. Ограничения версии v1	4
3.2. Описание алгоритма и функционирования программы	4
3.2.1. Архитектура узла	4
3.2.2. Подсистема хранения	5
3.2.3. MetadataFSM	5
3.2.4. PartitionFSM	5
3.2.5. Координаторы	6
3.2.6. gRPC API	6
3.3. Описание и обоснование выбора метода организации входных и выходных данных	6
3.3.1. Входные данные	6
3.3.2. Выходные данные	7
3.4. Описание и обоснование выбора состава технических и программных средств	7
3.4.1. Go 1.25	7
3.4.2. dragonboat v4	7
3.4.3. gRPC + Protocol Buffers	7
3.4.4. Prometheus	7
3.4.5. go.uber.org/zap	8
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	9
4.1. Показатели производительности	9
4.2. Показатели надежности	9
4.3. Показатели ресурсоемкости	9
4.4. Экономические показатели	10
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	11

1. ВВЕДЕНИЕ

Наименование программы – «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (в тексте – BunnyMQ).

Условное обозначение по ГОСТ 19.103-77: **RU.17701729.02-06**.

Основание разработки – техническое задание с тем же наименованием (обозначение: RU.17701729.02-06 ТЗ 01–1), принятое в 2026 г. на факультете компьютерных наук НИУ ВШЭ.

Исполнитель – студент программы «Программная инженерия», группа БПИ233, Б.А. Багавиев.

В состав изделия входят серверный процесс `bunnymq`, утилита командной строки `bunnymq-cli` и публичная библиотека `pkg/client`. Настоящая записка охватывает серверную часть: подсистему хранения, FSM Raft, координаторы и gRPC API.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

BunnyMQ – распределенный брокер сообщений, по архитектуре близкий к Apache Kafka: данные организованы по топикам и партициям, потребление идет по смещениям, однако вместо ISR-механизма используется Raft-репликация на уровне каждой партиции через dragonboat v4.

Три компонента системы:

- `bunmq` – серверный процесс, который принимает gRPC-запросы и пишет данные через Raft FSM в сегментированный журнал на диске;
- `bunmq-cli` – административный CLI для управления кластером и топиками;
- `pkg/client` – Go-библиотека общения с очередью сообщений с автоматическим поиском лидера и поддержкой групп консьюмеров.

2.2. Область применения

BunnyMQ рассчитан на backend-инфраструктуру, где между сервисами нужна очередь сообщений с гарантией порядка внутри партиции, с гарантией доставки сообщения и возможностью горизонтального масштабирования. Типичные сценарии – сбор телеметрии, конвейеры обработки событий, очереди задач.

Целевые пользователи – backend-разработчики, встраивающие `pkg/client` в прикладной код, и DevOps-инженеры, развертывающие кластер и администрирующие его через `bunmq-cli`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку

3.1.1. Описание задачи

Требуется реализовать распределенный брокер сообщений. Минимальный функциональный набор: управление топиками (создание, удаление, изменение числа партиций и политики хранения), публикация пакетов с выбором гарантии `acks=0` или `acks=all`, чтение по смещению с `long polling` при отсутствии данных, управление группами консьюмеров с диапазоным назначением партиций.

Должны поддерживаться корректное восстановление после обрыва записи (`torn write`), периодическая очистка данных по политике хранения (по `retention policy`), метрики в формате Prometheus, структурированное логирование.

3.1.2. Описание применяемых методов

Сегментированный журнал. Данные каждой партиции хранятся в виде последовательности сегментов. Сегмент – три файла с одинаковым именем (базовое смещение первого пакета, дополненное нулями до 20 знаков): `.log` с самими пакетами, `.index` с индексом смещений и `.timeindex` с временным индексом. Запись идет только в последний активный сегмент с флагами `O_APPEND|O_WRONLY`. Индексные файлы хранятся в mmap-отображениях, чтобы оптимизировать походы на диск и поиск в индексе; заполнение разрежено – индекс обновляется не на каждый пакет, а по достижении порогового числа байт. Поиск по смещению – бинарный по индексу, затем линейный по `.log`.

Формат пакета. Формат на диске совпадает с форматом на транспортном уровне, поэтому при передаче данных консьюмеру перекодирование не нужно. Совпадение форматов допущено намеренно для того, чтобы в будущем использовать оптимизацию `zero-copy` с помощью функции `sendfile()`, которая из файла данные пишет сразу на сетевую карту без копирования в `user-space`. Пакет начинается с 38-байтного заголовка, включающего базовое смещение, длину, число записей, контрольную сумму CRC-32C, атрибуты и два таймстампа; записи внутри кодируются через `varint`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Raft-репликация. Используется библиотека dragonboat v4 в схеме multi-raft: один shard на партицию плюс один shard для метаданных кластера. `acks=0` реализован через асинхронный `Propose`, который не дожидается подтверждения записи, `acks=all` – через блокирующий `SyncPropose`, который дожидается гарантированной записи на все реплики.

Детерминизм FSM. `MetadataFSM` и `PartitionFSM` не обращаются к системному времени и не генерируют случайных значений самостоятельно – все нефиксированные данные (таймстампы, идентификаторы участников групп) координатор передает в теле команды; без этого Raft-журнал нельзя корректно воспроизвести на репликах, так как для каждой реплики все должно быть одинаково и детерминированно.

Long polling. После каждого успешного `Append Storage` переключает канал уведомления `newDataCh`. При обработке `Fetch DataCoordinator` берет ссылку на текущий канал и пробует прочитать данные; если их нет – ждет закрытия канала или истечения `max_wait_ms`, а закрытие канала сигнализирует о новом пакете.

Range Assignment. При ребалансировке группы партиции и участники сортируются по идентификаторам, затем нарезаются на последовательные диапазоны. Первые $P \bmod M$ участников получают на одну партицию больше, остальные – базовый размер. Алгоритм совпадает с реализацией в Apache Kafka.

Torn write recovery. При каждом `Append FSM` записывает примененный индекс Raft в файл `applied.idx`, а при запуске `Open` сравнивает реальное состояние журнала с контрольной точкой и отбрасывает хвост, если обнаруживает расхождение.

3.1.3. Ограничения версии v1

`acks=1` намеренно не реализован – при Raft это не нужно, есть `acks=0` для скорости и `acks=all` для надежности. Также снимки dragonboat отключены, потому что FSM восстанавливается воспроизведением полного журнала. Состав кластера задается в конфигурации при запуске и не меняется в рантайме. Транзакционная запись как в Kafka тоже пока не поддерживается.

3.2. Описание алгоритма и функционирования программы

3.2.1. Архитектура узла

Каждый узел – один процесс `bunnymq` с несколькими слоями в коде:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

cmd/bunnymq/
└─ main.go -- инициализация и graceful shutdown
internal/
├─ storage/ -- сегментированный журнал
├─ partition/ -- PartitionFSM (IOOnDiskStateMachine)
├─ metadata/ -- MetadataFSM (IStateMachine)
├─ raft/ -- обертка NodeHost
├─ coordinator/
│   ├── cluster/ -- жизненный цикл shards и смена лидера
│   ├── data/ -- маршрутизация Produce/Fetch
│   └─ group/ -- протокол групп консьюмеров
└─ api/
    ├── data/ -- gRPC DataService (порт 9090)
    └─ management/ -- gRPC ManagementService (порт 9091)

```

Схема взаимодействия компонентов приведена в Приложении 1.

3.2.2. Подсистема хранения

Storage ведет упорядоченный набор сегментов. При Append проставляет `base_offset` в заголовок пакета, передает пакет активному сегменту, при необходимости запечатывает его и создает новый. После записи переключает `newDataCh`.

Read находит нужный сегмент бинарным поиском по набору, локализует позицию через `.index`-файл и читает пакеты линейно до исчерпания `maxBytes`.

Фоновая горютина раз в минуту проверяет сегменты на соответствие политике хранения по времени и объему, но активный сегмент не удаляется

3.2.3. MetadataFSM

Хранит состояние кластера в памяти (топики, партии, группы консьюмеров, узлы), снимок – JSON-сериализация всего состояния. Чтение идет через прямой Lookup, запись – только через Raft, а команды упакованы в JSON-конверт `MetadataCommand`.

3.2.4. PartitionFSM

Управляет одной репликой одной партии, единственная команда – `AppendBatch`: вызывает `Storage.Append`, синхронизирует данные на диск, об-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

новляет `applied.idx`. При открытии сравнивает реальное состояние журнала с контрольной точкой и усекает хвост при расхождении.

3.2.5. Координаторы

`ClusterCoordinator` запускает `shards dragonboat` и отслеживает смену лидера – при смене обновляет соответствующие поля в `MetadataFSM`.

`DataCoordinator` маршрутизирует запросы `Data API`: если текущий узел – лидер нужной партии, обрабатывает запрос локально; иначе возвращает `NotLeader` с адресом реального лидера. Для `Fetch` реализует `long polling`.

`GroupCoordinator` управляет группами консьюмеров через `MetadataFSM`: `JoinGroup` генерирует `member_id` на стороне сервера и добавляет участника, а фоновая горутинка каждые 5 секунд исключает участников с истекшим `session timeout`.

3.2.6. gRPC API

`ManagementService` (на порте 9091) и `DataService` (на порте 9090) – два независимых `gRPC`-сервера. Перед каждым методом `auth interceptor` проверяет заголовок `bunnumq-auth-token` при наличии токенов в конфигурации. Метрики `Prometheus` доступны на порту 9092.

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Входные данные

Все клиентские взаимодействия – через `gRPC` поверх `HTTP/2`, а схемы запросов описаны `.proto`-файлами пакета `bunnumq.v1`. Пакеты сообщений передаются в сыром байтовом формате, совпадающем с форматом хранения.

Конфигурация узла задается `YAML`-файлом (флагом `--config`) и загружается при старте однократно и при работе программы не меняются; `SIGTERM` и `SIGINT` инициируют `graceful shutdown`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3.2. Выходные данные

gRPC-ответы в формате Protobuf. При Fetch пакеты передаются в поле `records bytes` без перекодирования – та же байтовая последовательность, что хранится на диске. Метрики – `text/plain` в формате OpenMetrics по HTTP. Логи – JSON на `stdout`.

3.4. Описание и обоснование выбора состава технических и программных средств

3.4.1. Go 1.25

Горутинная модель Go справляется с сотнями одновременных gRPC-соединений без потока на каждое; GC с малыми паузами не вносит неконтролируемых задержек на горячем пути записи, а статическая компиляция без внешних зависимостей упрощает развертывание.

3.4.2. dragonboat v4

Единственная Go-реализация Raft с поддержкой multi-raft из коробки – это принципиально важно: сбой одного shard не блокирует другие. Через `IONDiskStateMachine PartitionFSM` работает с внешним хранилищем напрямую, минуя snapshot-механизм dragonboat. Альтернативы (etcd/raft, hashicorp/raft) multi-raft не поддерживают и требуют значительно большего объема интеграционного кода.

3.4.3. gRPC + Protocol Buffers

Строгая типизация proto-схем исключает несогласованность между клиентом и сервером. Бинарный формат Protobuf компактнее JSON при передаче пакетов сообщений, а HTTP/2 снижает накладные расходы при параллельных запросах. Формат хранения на диске совпадает с форматом передачи – данные консьюмеру уходят без перекодирования.

3.4.4. Prometheus

dragonboat экспортирует метрики в глобальный реестр Prometheus при включении `EnableMetrics`. Pull-модель не требует изменений в сервисе при добавлении scrapers.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.4.5. go.uber.org/zap

Собирает JSON-строки без рефлексии – на горячем пути записи это важно, прочие логи добавляли бы измеримые накладные расходы. Уровень логирования задается в конфигурационном файле

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Показатели производительности

Оценки получены на основании бенчмарков дисковой записи (benchmarks/disk_test.go) и характеристик dragonboat v4.

Таблица 4.1 — Ожидаемые показатели производительности BunnyMQ

Показатель	Значение	Примечание
Запись (acks=all, 1 партиция)	≥ 50 МБ/с	Ограничена диском и задержкой Raft
Запись (acks=0, 1 партиция)	≥ 200 МБ/с	Ограничена только диском
Задержка Produce (acks=all, p99)	≤ 50 мс	Два RTT Raft + fsync
Задержка Produce (acks=0, p99)	≤ 5 мс	Без ожидания кворума
Задержка Fetch (есть данные, p99)	≤ 10 мс	Чтение с диска без репликации
Уведомление при long-poll	≤ 5 мс	Через newDataCh после Append
Минимальный размер кластера	3 узла	Raft-кворум: 2 из 3
Максимальный пакет	1 МиБ	Ограничение gRPC по умолчанию

4.2. Показатели надежности

Отказ одного узла из трех не нарушает доступность – кворум сохраняется. Torn write исправляется автоматически при перезапуске за счет сравнения applied.idx с реальным состоянием журнала. Минимальная единица восстановления – целый пакет; частично записанные пакеты отбрасываются.

4.3. Показатели ресурсоемкости

Таблица 4.2 — Минимальные требования к ресурсам узла

Ресурс	Требование
CPU	2 ядра x86-64; 4 рекомендовано при нагрузке более 10 000 RPS
RAM	512 МБ без учета mmap-индексов; индексы вытесняются ОС при нехватке памяти
Диск	SSD рекомендован; объем – retention_bytes × число партиций на узле плюс журналы Raft
Сеть	TCP-связность между всеми узлами; задержка менее 100 мс

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.4. Экономические показатели

Разработка выполнена в рамках учебного проекта. Все используемые библиотеки распространяются под открытыми лицензиями (Apache 2.0, MIT, BSD). Для запуска трех узлов достаточно трех виртуальных машин стандартной конфигурации, стоимостью порядка 1 500–3 000 руб./мес. каждая в публичных облаках.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
2. ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам. — М.: Стандартинформ, 2010.
3. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом. — М.: Стандартинформ, 2010.
4. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
5. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
6. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).
7. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
8. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
9. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
10. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
11. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

12. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).
13. Uber go/zip. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zip> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1**ОПИСАНИЕ ПОТОКОВ ДАННЫХ УЗЛА**

Входящий gRPC-запрос от клиента проходит через auth interceptor, после чего попадает в ManagementService (9091) или DataService (9090).

ManagementService передает команды в ClusterCoordinator, который пишет их в MetadataFSM через raft.Host (shard 0).

DataService передает запросы в DataCoordinator. При публикации DataCoordinator проверяет, лидер ли текущий узел для нужной партиции, и либо обрабатывает запрос локально, либо возвращает адрес лидера. PartitionFSM пишет данные в Storage; после каждого успешного Append Storage переключает newDataCh – висящие Fetch-запросы разблокируются.

GroupCoordinator обрабатывает операции групп консьюмеров через MetadataFSM.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ФОРМАТ ПАКЕТА ДАННЫХ

Таблица П2.1 — Заголовок пакета (38 байт, big-endian)

Смещение	Длина	Тип	Поле
0	8	int64	base_offset – базовое смещение первой записи; заполняется Storage
8	4	int32	batch_length – полная длина пакета в байтах
12	4	int32	record_count – число записей в пакете
16	4	uint32	crc32 – CRC-32C над bytes[38..batch_length)
20	2	int16	attributes – зарезервировано (0 в v1)
22	8	int64	base_timestamp – время первой записи, мс от Unix-эпохи
30	8	int64	max_timestamp – время последней записи, мс
38	var	[]byte	records – varint-кодированные записи

Таблица П2.2 — Запись индекса смещений (.index, 8 байт)

Байты	Тип	Поле
0–3	int32	relative_offset = base_offset – segment.base_offset
4–7	int32	position – смещение пакета в .log-файле

Таблица П2.3 — Запись временного индекса (.timeindex, 12 байт)

Байты	Тип	Поле
0–7	int64	timestamp_ms = batch.max_timestamp
8–11	int32	relative_offset

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]